

Spiegazioni Dettagliate – Sistemi Intelligenti

Tutte le domande di esame con risposta approfondita
Crocette, Vero/Falso, Aperte e Orali

A.A. 2025–2026

Indice

1	Intelligenza Artificiale: Basi	3
1.1	Test di Turing	3
1.2	IA Forte vs IA Debole	3
2	Ricerca nello Spazio degli Stati	3
2.1	Componenti di un problema di ricerca	3
2.2	Algoritmi di ricerca: confronto	4
2.3	Euristiche	4
3	CSP – Problemi di Soddisfacimento di Vincoli	5
3.1	Definizione	5
3.2	Algoritmi per CSP – efficienza comparata	5
3.3	Back-jumping e Conflict Set	5
3.4	Path Consistency	6
3.5	Forward Checking	6
4	Ricerca Avversariale	6
4.1	Giochi a somma zero	6
4.2	Funzionamento di Minimax	6
5	Agenti Logici e Logica Proposizionale	7
5.1	Tautologie, Contraddizioni, Soddisfacibilità	7
5.2	Forma Normale Congiuntiva (CNF)	7
5.3	Clausole di Horn e Forward Chaining	8
6	Logica del Primo Ordine (FOL)	9
6.1	Funzioni di Skolem e termini ground	9
6.2	Risoluzione in FOL	9

6.3	Analisi sistematica degli esercizi V/F su FOL	9
7	Machine Learning	10
7.1	Alberi di Decisione	10
7.2	Entropia e Information Gain	10
7.3	Overfitting e Pre/Post-Pruning	11
7.4	Percettrone	12
8	Matrice di Confusione	12
9	Ontologie	12
10	Frame Problem e Situation Calculus	13
11	Associazioni – Schema Completo	14

1 Intelligenza Artificiale: Basi

1.1 Test di Turing

Risposta: Il test di Turing dice che un computer è intelligente se...

imita il comportamento umano nel rispondere alle domande.

Il test (1950) prevede un giudice umano che conversa via tastiera con due interlocutori: un umano e una macchina. Se il giudice non riesce a distinguerli, la macchina supera il test.

Perché le altre risposte sono sbagliate:

- “Comprende le domande” $\Rightarrow$ la comprensione semantica **non è verificabile** dal test (argomento della Stanza Cinese di Searle).
- “Risponde correttamente” $\Rightarrow$ sbagliato: non servono risposte esatte, serve **sembrare umano** (un umano sbaglia spesso).

1.2 IA Forte vs IA Debole

IA Debole (Weak AI)

La macchina *simula* l'intelligenza per compiti specifici. Non ha comprensione reale, coscienza o stati mentali. Esempio: chatbot, sistemi esperti, motori di scacchi.

IA Forte (Strong AI)

La macchina possiede vera comprensione, coscienza e stati mentali equivalenti a quelli umani. È un obiettivo teorico, non ancora raggiunto.

Argomento della Stanza Cinese (Searle): seguire regole sintattiche non implica comprensione semantica. Un operatore che traduce simboli cinesi seguendo un manuale non *capisce* il cinese.

2 Ricerca nello Spazio degli Stati

2.1 Componenti di un problema di ricerca

Stato iniziale, Funzione Successore, Test Obiettivo

- **Stato iniziale** s_0 : configurazione di partenza del problema.
- **Funzione successore** (modello di transizione): $\text{Result}(s, a)$ – dato uno stato s e un'azione a , restituisce il nuovo stato.
- **Test obiettivo** $\text{IsGoal}(s)$: verifica se lo stato corrente è una soluzione.
- **Costo del cammino**: somma dei costi delle azioni intraprese.

Una **soluzione** è una sequenza di azioni che porta da s_0 a uno stato obiettivo.

2.2 Algoritmi di ricerca: confronto

Tabella complessità – da memorizzare

Algoritmo	Completo	Ottimale	Tempo	Spazio
BFS	Sì	Sì*	$O(b^{d+1})$	$O(b^{d+1})$
DFS	No	No	$O(b^m)$	$O(b \cdot m)$
IDDFS	Sì	Sì*	$O(b^d)$	$O(b \cdot d)$
UCS	Sì	Sì	$O(b^{1+C^*/\varepsilon})$	uguale
Greedy	No	No	$O(b^m)$	$O(b^m)$
A*	Sì	Sì	$O(b^d)$	$O(b^d)$

*Con costi uniformi. b =branching factor, d =profondità soluzione, m =profondità massima.

Risposta: Complessità spaziale minore: DFS e IDDFS

DFS ha complessità spaziale $O(b \cdot m)$: mantiene solo il *cammino corrente* dalla radice al nodo espanso + i fratelli non ancora esplorati.

IDDFS ha $O(b \cdot d)$: combina il vantaggio spaziale di DFS con la completezza e ottimalità di BFS. Per questo è considerato il **miglior algoritmo blind** in generale.

BFS e A* sono i peggiori come spazio ($O(b^d)$): mantengono tutta la frontiera in memoria.

2.3 Euristiche

Euristica ammissibile

$h(n)$ è ammissibile se $h(n) \leq h^*(n)$ per ogni nodo n , dove $h^*(n)$ è il costo **reale ottimo** per raggiungere il goal da n .

Non sovrastima mai il costo $\Rightarrow$ è “ottimistica”. A* con euristica ammissibile è ottimale su alberi.

Euristica consistente (monotona)

$h(n) \leq c(n, n') + h(n')$ per ogni arco (n, n') .

Consistente $\Rightarrow$ Ammissibile (non vale il contrario). A* con euristica consistente è ottimale anche su grafi (con lista chiusa).

Euristica dominante

h_1 **domina** h_2 se $h_1(n) \geq h_2(n)$ per **ogni** nodo n .

Dominante = più informativa = A* espande meno nodi = più efficiente. La distanza di Manhattan domina i pezzi fuori posto nell'8-puzzle.

Funzione costante zero: $h(n) = 0$ è ammissibile ma totalmente non informativa. A* degenera in UCS, non guida verso il goal.

Euristiche MRV e di Grado (per CSP)

- **MRV (Minimum Remaining Values):** scegliere la variabile con il **dominio più piccolo** – *fail-first*, scopre i fallimenti prima.

- **Euristica di grado:** tra variabili con MRV uguale, scegliere quella con **più vincoli** sulle variabili non ancora assegnate $\Rightarrow$ identifica la variabile coinvolta in più vincoli.
- **Euristica monotona:** garantisce l'ottimalità nella ricerca su grafo con lista chiusa.

3 CSP – Problemi di Soddisfacimento di Vincoli

3.1 Definizione

CSP – tripla $\langle X, D, C \rangle$

- $X = \{X_1, \dots, X_n\}$: variabili
- $D = \{D_1, \dots, D_n\}$: domini (valori possibili per ciascuna variabile)
- $C = \{C_1, \dots, C_m\}$: vincoli

Assegnamento consistente: non viola alcun vincolo.

Assegnamento completo: tutte le variabili hanno un valore.

Soluzione CSP: assegnamento **completo E consistente**.

Attenzione: differenza tra soluzione CSP e soluzione di ricerca

Nella ricerca in spazio degli stati, la soluzione è una *sequenza di azioni* a costo minimo.

Nel CSP, la soluzione è un *assegnamento* (non una sequenza di passi): non si minimizza un costo, si soddisfano i vincoli.

3.2 Algoritmi per CSP – efficienza comparata

Risposta: Ordine di efficienza (dal meno al più efficiente)

1. **Generate and test** (PEGGIORE): genera tutti gli assegnamenti completi, poi verifica. Complessità $O(d^n)$.
2. **Backtracking con commutatività:** migliore, evita permutazioni equivalenti.
3. **Back-jumping** (MIGLIORE): al fallimento, torna direttamente alla variabile responsabile (conflict set), evitando backtrack inutili.

Backward chaining non è un algoritmo per CSP: è per la FOL (logica).

3.3 Back-jumping e Conflict Set

Back-jumping

Quando una variabile X_k fallisce (il suo dominio è vuoto), invece di tornare alla variabile precedente (X_{k-1}), si torna alla variabile più recente nel **conflict set** di X_k :

$$CS(X_k) = \{X_j : j < k, \exists \text{ vincolo tra } X_j \text{ e } X_k\}$$

Il back-jumping **salta** tutte le variabili intermedie che non hanno causato il conflitto, riducendo drasticamente il numero di backtrack.

3.4 Path Consistency

Gerarchia di consistenza

- **1-consistenza (node)**: ogni valore nel dominio soddisfa i vincoli unari.
- **2-consistenza (arc)**: per ogni valore di X_i , esiste un valore compatibile in X_j .
- **3-consistenza (path)**: per ogni coppia (X_i, X_j) consistente, esiste un valore per X_m che soddisfa i vincoli con entrambe.

La **path consistency** considera **triple** di variabili (non coppie come arc consistency).

3.5 Forward Checking

Forward Checking

Dopo ogni assegnamento, si eliminano dai domini delle variabili *non ancora assegnate* i valori incompatibili con l'assegnamento appena fatto. Se un dominio si svuota $\Rightarrow$ backtrack immediato.

MAC (Maintaining Arc Consistency): versione più potente che applica AC-3 completo dopo ogni assegnamento. Pota di più ma è computazionalmente più costoso.

4 Ricerca Avversariale

4.1 Giochi a somma zero

Gioco a somma zero – definizione

Un gioco è a **somma zero** se il guadagno di un giocatore corrisponde esattamente alla perdita dell'altro: $Utility(MAX) + Utility(MIN) = \text{costante}$.

Caratteristiche: **informazione perfetta** (entrambi vedono lo stato completo), **deterministico**, **due giocatori** (MAX e MIN), **a turni**.

Esempi: tris, scacchi, dama, go.

4.2 Funzionamento di Minimax

Algoritmo Minimax – come funziona

1. Si espande l'albero di gioco fino alle foglie (o alla profondità limite).
2. Le foglie ricevono il valore dalla **funzione di utilità** (stati terminali) o dalla **funzione di valutazione EVAL** (profondità limite).
3. Si propagano i valori **dal basso verso l'alto** (dalle foglie alla radice):
 - Nodi MAX: scelgono il **massimo** tra i figli.
 - Nodi MIN: scelgono il **minimo** tra i figli.
4. La radice indica la mossa ottimale per MAX.

Minimax suggerisce la prossima mossa, non costruisce un percorso verso la vittoria: MAX potrebbe non vincere se l'avversario è molto forte o la posizione è già perdente.

Formula Minimax

$$\text{MiniMax}(s) = \begin{cases} \text{Utility}(s) & \text{se } s \text{ è terminale} \\ \max_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MAX} \\ \min_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MIN} \end{cases}$$

Complessità: Tempo $O(b^m)$, Spazio $O(b \cdot m)$. Con alpha-beta pruning (caso migliore): $O(b^{m/2})$.

5 Agenti Logici e Logica Proposizionale

5.1 Tautologie, Contraddizioni, Soddisfacibilità

Classificazione delle formule logiche

- **Tautologia (formula valida):** vera in **tutti** i modelli. Es: $A \vee \neg A$.
- **Contraddizione (insoddisfacibile):** falsa in **tutti** i modelli. Es: $A \wedge \neg A$.
- **Soddisfacibile:** vera in **almeno un** modello. SAT è NP-completo.
- **Contingente:** né tautologia né contraddizione (dipende dal modello).

Teorema di deduzione: $KB \models \alpha \Leftrightarrow (KB \Rightarrow \alpha)$ è valida (tautologia).

Refutazione: $KB \models \alpha \Leftrightarrow KB \wedge \neg \alpha$ è insoddisfacibile (contraddizione).

5.2 Forma Normale Congiuntiva (CNF)

CNF e conversione

La **CNF** è una congiunzione di **clausole**, dove ogni clausola è una **disgiunzione di letterali**.

Es: $(A \vee \neg B) \wedge (B \vee C \vee \neg A)$

Conversione in CNF:

1. Eliminare $\Leftrightarrow$: $\alpha \Leftrightarrow \beta \rightarrow (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
2. Eliminare $\Rightarrow$: $\alpha \Rightarrow \beta \rightarrow \neg \alpha \vee \beta$
3. Portare $\neg$ verso l'interno (De Morgan): $\neg(\alpha \wedge \beta) \rightarrow \neg \alpha \vee \neg \beta$

4. Distribuire $\vee$ su $\wedge$: $(A \vee (B \wedge C)) \rightarrow (A \vee B) \wedge (A \vee C)$

Serve per applicare la **risoluzione**.

5.3 Clausole di Horn e Forward Chaining

Clausole di Horn – definizione

Una clausola con **al più un letterale positivo** (non negato).

Forma implicativa: $P_1 \wedge P_2 \wedge \dots \wedge P_k \Rightarrow Q$

Tipi:

- **Clausola definita:** esattamente un letterale positivo (regola con testa).
- **Clausola obiettivo/fatto:** zero letterali positivi (solo negazioni = goal negato).
- **Fatto:** letterale positivo senza premesse (es. Q da solo).

Importante: l'inferenza su clausole di Horn è decidibile in **tempo polinomiale**.

Come riconoscere clausole di Horn – trucco rapido

Scrivi la clausola come disgiunzione di letterali e conta i letterali **positivi** (non negati):

- $\neg P(x) \vee Q(x) \vee \neg R(x)$: un solo positivo (Q) $\Rightarrow$ **Horn** ✓
- $P(x) \vee Q(x) \vee \neg R(x)$: due positivi (P, Q) $\Rightarrow$ **NON Horn** ×
- $\neg P(x) \vee \neg Q(x)$: zero positivi $\Rightarrow$ **Horn** ✓ (clausola obiettivo)

Forward Chaining (concatenazione in avanti)

Algoritmo:

1. Parti dai fatti noti nella KB.
2. Per ogni regola di Horn le cui premesse sono tutte soddisfatte, aggiungi la conclusione ai fatti noti.
3. Ripeti fino al *punto fisso* (nessun nuovo fatto deducibile) o fino a trovare il goal.

Guidato dai dati. Corretto e completo per clausole definite.

Richiede: (1) la clausola sia di Horn, (2) **tutti** i fatti nelle premesse siano noti in KB.

Modus Ponens Generalizzato (MPG): versione lifted del forward chaining con unificazione.

Modus Ponens – formula

$$\frac{\alpha \quad \alpha \Rightarrow \beta}{\beta}$$

In FOL (MPG):

$$\frac{p_1, p_2, \dots, p_k \quad (P_1 \wedge \dots \wedge P_k \Rightarrow Q)[\theta]}{Q\theta}$$

dove θ è la sostituzione che unifica P_i con p_i .

6 Logica del Primo Ordine (FOL)

6.1 Funzioni di Skolem e termini ground

Termini ground

Un termine è **ground** (senza variabili) se contiene solo costanti e funzioni applicate a costanti.

Es: Padre(Giovanni), Anna, Proprietario(Asso) $\Rightarrow$ ground.

Es: Padre(x), $P(y)$ $\Rightarrow$ **non** ground (contengono variabili).

Una formula è **ground** se non contiene variabili libere.

Funzioni di Skolem – skolemizzazione

Nella conversione in CNF per FOL, si elimina il quantificatore esistenziale $\exists$:

- $\exists x P(x)$ (senza $\forall$ esterno): si sostituisce x con una **costante di Skolem** k .
- $\forall y \exists x P(x, y)$ ($\exists$ sotto $\forall$): si sostituisce x con una **funzione di Skolem** $f(y)$.

Perché? I quantificatori universali sono impliciti in CNF. La skolemizzazione preserva la soddisfacibilità (non la validità).

Esempio: $\forall y \exists x \text{Madre}(x, y) \rightarrow \text{Madre}(m(y), y)$ dove m è la funzione di Skolem.

6.2 Risoluzione in FOL

Risoluzione (Binary Resolution)

Date due clausole con letterali complementari (uno è la negazione dell'altro), la **risoluzione** produce il **risolvente**: la clausola ottenuta eliminando i letterali complementari.

In FOL: si usa l'**unificatore** per rendere complementari i letterali.

Es: $(A \vee C)$ e $(\neg C \vee B) \rightarrow$ risolvente $(A \vee B)$.

Dimostrazione per refutazione: per dimostrare $KB \models \alpha$:

1. Si aggiunge $\neg\alpha$ alla KB.
2. Si applicano ripetutamente le risoluzioni.
3. Se si deriva la **clausola vuota** $\square \Rightarrow$ contraddizione $\Rightarrow KB \models \alpha$.

6.3 Analisi sistematica degli esercizi V/F su FOL

Schema per rispondere agli esercizi V/F

Data una formula A (clausola) e formule B, C (fatti):

Domanda 1 – “A è una clausola?”

Quasi sempre VERO: qualsiasi disgiunzione di letterali è una clausola. Verificare solo che sia scritta in forma normale (disgiunzione).

Domanda 2 – “A è una clausola di Horn?”

Conta i letterali positivi in A:

- 0 o 1 positivi $\Rightarrow$ VERO (Horn)
- 2+ positivi $\Rightarrow$ FALSO (non Horn)

Domanda 3 – “La risoluzione è applicabile?”

VERO se esiste almeno una coppia di letterali complementari tra A e le altre formule (con unificazione). Non richiede Horn.

Domanda 4 – “Il forward chaining è applicabile?”

VERO solo se:

1. A è una clausola di Horn (al più un letterale positivo)
2. **Tutti** i fatti nelle premesse di A sono presenti in KB (come B, C, ...)

Domanda 5 – “Il MPG è applicabile?”

Stesse condizioni del forward chaining.

Esempio: Caso tipico all'esame: NOT Alfa(x) or Beta(x) or NOT Gamma(x)

A) $\neg \text{Alfa}(x) \vee \text{Beta}(x) \vee \neg \text{Gamma}(x)$ [equivale a: $\text{Alfa}(x) \wedge \text{Gamma}(x) \Rightarrow \text{Beta}(x)$]

B) Alfa(ID0) C) Gamma(ID0)

- A è clausola: **VERO** (disgiunzione di letterali)
- A è clausola di Horn: **VERO** (un solo positivo: Beta(x))
- Risoluzione applicabile: **VERO** ($\neg \text{Alfa}(x)$ si unifica con Alfa(ID0))
- FC applicabile: **VERO** se B e C entrambi in KB; **FALSO** se manca uno dei due
- MPG applicabile: stessa risposta del FC

Con sostituzione $\{x/\text{ID0}\}$ e B, C: si deriva **Beta(ID0)**.

7 Machine Learning

7.1 Alberi di Decisione

Costruzione dell'albero di decisione (Algoritmo di Hunt / ID3)

1. Se tutte le istanze nel nodo appartengono alla stessa classe $\Rightarrow$ **foglia**.
2. Se non ci sono istanze $\Rightarrow$ foglia con classe di default.
3. Altrimenti: scegli l'attributo che massimizza il **guadagno di informazione**.
4. Crea un nodo figlio per ogni valore dell'attributo scelto.
5. Applica ricorsivamente ai figli.

Criterio di terminazione (pre-pruning): numero di istanze nel nodo < soglia, oppure guadagno < soglia, oppure profondità massima raggiunta.

7.2 Entropia e Information Gain

Entropia e Information Gain – formule

Entropia (misura di impurezza/disordine):

$$H(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$H = 0$: nodo puro (una sola classe). $H = 1$: massima confusione (2 classi equiprobabili).

Cosa misura l'entropia: la presenza di più classi in un insieme di dati (impurezza).

Information Gain:

$$\text{Gain}(A) = H(\text{padre}) - \sum_{v \in \text{valori}(A)} \frac{|D_v|}{|D|} \cdot H(D_v)$$

Cosa misura l'IG: la **riduzione di entropia** prodotta dallo split sull'attributo A . Si sceglie l'attributo con **Gain massimo**.

Attenzione: entropia $\neq$ information gain

- **Entropia** misura l'**impurezza** di un nodo (quanto sono mescolate le classi).
- **Information Gain** misura la **riduzione di entropia** dopo uno split.
- Entrambe si calcolano **prima della matrice di confusione** (sul training set, non sulle predizioni).
- La **matrice di confusione** calcola accuratezza, error rate, precision, recall – **non** entropia né IG.

7.3 Overfitting e Pre/Post-Pruning

Overfitting

Il modello si adatta **troppo bene** al training set (memorizza anche il rumore) e **generalizza male** sui dati nuovi.

Cause: troppi parametri, pochi dati, rumore nel dataset.

Sintomo: accuratezza alta sul training, bassa sul test.

Principio di Occam: a parità di errore, preferire il modello **più semplice**.

Pre-Pruning vs Post-Pruning

Pre-Pruning:

- Si interrompe la costruzione *prima* del completamento.
- Criteri: numero di istanze $<$ soglia, guadagno $<$ soglia, profondità massima.
- Rischio: underfitting (si ferma troppo presto).

Post-Pruning:

- Si costruisce l'albero completo, poi si *potano* i rami che peggiorano le prestazioni sul validation set.
- Si sostituisce un sottoalbero con una foglia (classe più frequente nelle sue foglie).
- Contesto: **apprendimento automatico** (non CSP, non giochi).

7.4 Percettrone

Percettrone (Rosenblatt, 1957)

Singolo neurone artificiale: $y = f(\mathbf{w} \cdot \mathbf{x} + b)$.

Regola di apprendimento (Regola Delta):

$$w_j(t+1) = w_j(t) + \eta \cdot (d - o) \cdot x_j$$

dove η è il learning rate, d è il valore desiderato, o è l'output.

Problema rilevante: OR esclusivo (XOR).

Il percettrone non può imparare XOR perché **non è linearmente separabile**. Minsky e Papert (1969) lo dimostrarono, portando al primo “inverno dell'IA”.

Base di K-NN vs percettrone:

- K-NN: classificazione basata sulla **somiglianza** (senza modello, lazy).
- Percettrone: apprende un **modello** (iperpiano separatore) dal training set.

8 Matrice di Confusione

Matrice di confusione – struttura e metriche

Per classificazione binaria (classi: Positivo/Negativo):

	Predetto +	Predetto –
Reale +	TP (Vero Positivo)	FN (Falso Negativo)
Reale –	FP (Falso Positivo)	TN (Vero Negativo)

Accuratezza: $\frac{TP + TN}{TP + TN + FP + FN}$

Error rate: $1 - \text{Accuratezza}$

Prestazione ottimale: tutta la diagonale principale (TP e TN), zero fuori dalla diagonale.

Cosa NON si calcola: entropia, information gain, overfitting, complessità del modello.

9 Ontologie

Relazioni nelle ontologie – riassunto

- **isa (is-a): relazione di sottoclasse.** “Virologo isa microbiologo” = virologo è una sottoclasse di microbiologo. Eredita le proprietà della superclasse.
- **part-of: relazione di composizione.** “Capsomero part-of virus” = il capsomero è una parte del virus.
- **member: relazione di istanza.** “HIV member virus” = HIV è un'istanza (esemplare specifico) della classe virus.
- **member(X, C) $\Rightarrow$ P(X): proprietà di classe.** Ogni membro della classe C ha la proprietà P.
- **Partizione:** insieme di sottocategorie disgiunte ed esaustive rispetto alla

sovracategoria.

- **Decomposizione esaustiva:** le sottocategorie coprono tutti i casi (ma non necessariamente disgiunte).

Tassonomia – affermazione corretta

In una tassonomia: “**Le sottocategorie di ciascuna categoria costituiscono una partizione**”.

Cioè, per ogni categoria C : le sue immediate sottocategorie sono disgiunte tra loro ed esaustive rispetto a C .

Sbagliato: “L’insieme di tutte le categorie è una partizione” (non è vero: le categorie possono sovrapporsi tra livelli diversi).

RDF (Resource Description Framework)

RDF rappresenta la conoscenza come **triple** $\langle \text{oggetto}, \text{predicato}, \text{oggetto} \rangle$.

Es: $\langle \text{HIV}, \text{isa}, \text{Virus} \rangle$, $\langle \text{Roma}, \text{capitaleOf}, \text{Italia} \rangle$.

Non usa clausole di Horn (quelle sono per la logica proposizionale/FOL), non solo IS-A e MEMBER.

10 Frame Problem e Situation Calculus

Frame Problem – definizione precisa

Il **frame problem** è l’incapacità di sapere quali proprietà di una situazione si **preservano** (non cambiano) dopo l’applicazione di un’azione.

Problema: se sposto un oggetto, devo specificare esplicitamente che tutto il resto rimane uguale? Senza assiomi del frame, questo richiederebbe infinite formule.

Soluzione nel Situation Calculus: assiomi del successore di stato: “Un fluente F è vero in $\text{Result}(a, s)$ se e solo se a lo ha reso vero, oppure era già vero e a non lo ha reso falso.”

Non confondere con:

- I limiti del perceptron (XOR) – quello è il *problema dell’OR esclusivo*.
- L’anomalia di Sussman – quella riguarda la pianificazione non-lineare.

Situation Calculus – definizione di azione

Un’azione nel Situation Calculus è definita da:

1. **Precondizioni:** proprietà che devono valere per applicare l’azione.
2. **Effetti:** proprietà che l’applicazione dell’azione modifica.

NON fa parte della definizione di azione: le “proprietà atemporali” (quelle sono fatti permanenti della KB, non di una specifica azione).

11 Associazioni – Schema Completo

Associazioni algoritmi → compiti (da sapere a memoria)

Algoritmo/meccanismo	Compito
Model checking	conseguenza logica
Modus Ponens (MP)	forward chaining proposizionale
Modus Ponens Generalizzato (MPG)	forward chaining FOL
Risoluzione	refutazione proposizionale
Binary resolution	refutazione FOL
Alpha-beta pruning	riduzione spazio di ricerca in gioco con avversario
Minimax	suggerimento prossima mossa con avversario
RBFS	ricerca informata in spazio degli stati
k-NN	classificazione senza un modello
Algoritmo di Hunt	costruzione alberi di decisione
Back-jumping	risoluzione CSP
Forward chaining (logica)	inferenza da KB in clausole di Horn
Forward checking (CSP)	ricerca soluzione per un CSP
Ricerca bidirezionale	ricerca soluzione in spazio degli stati